Rapport Banque 10

Ce rapport analyse l'octroi d'un **prêt personnel** à partir du jeu de données **Banque10** (n=5000) et construit un score de probabilité pour de nouveaux clients. Au-delà de la comparaison de modèles de classification, l'objectif est d'apporter une **lecture économique** : profils des clients acceptés, variables discriminantes, interprétation des coefficients et discussion sur le **coût du risque** (faux positifs vs faux négatifs). Le critère principal de comparaison est l'**AUC ROC**, complété par des métriques au seuil 0.5 (Accuracy, Precision, Recall).

RAPPORT ML

JANVIER 2026

Antoine Jean
Laury hildevert

1 Introduction

L'octroi de crédit est une décision sous contrainte : la banque recherche des marges (revenus d'intérêt) tout en maîtrisant le risque (défaut, provisions, capital réglementaire). Dans ce cadre, un modèle de scoring vise à fournir une probabilité d'octroi (ou d'éligibilité) exploitable dans un processus de décision, avec deux exigences : la performance (bien discriminer les clients) et la gouvernance (justifier la décision).

Ce rapport s'appuie sur le jeu de données Banque10 (n=5000), composé d'informations sociodémographiques et financières. L'objectif est d'identifier le profil associé à l'octroi, de comparer plusieurs modèles de classification, et de proposer une recommandation cohérente avec un contexte bancaire. La comparaison repose principalement sur l'AUC ROC, complétée par l'Accuracy, la Precision et le Recall au seuil 0.5.

2 Données et préparation

2.1 Variables

Les variables explicatives sont : Age, Experience, Revenu (milliers), Dépense moyenne (milliers), Famille, Education et Compte Épargne Logement (CEL). La cible Prêt Personnel vaut 1 si le prêt a été accordé.

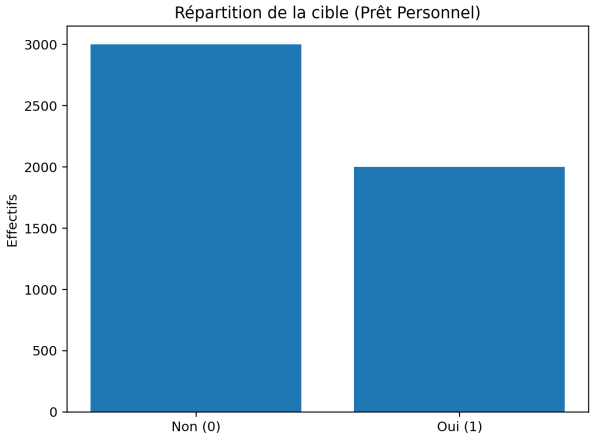

Figure 1 – *Répartition de la variable cible (taux d'octroi 40%).*

Le taux d'octroi observé est de 40%. Le déséquilibre est modéré (60/40), ce qui est important pour l'analyse des méthodes dites balanced.

2.2 Pré-traitements

— Recodage du CEL (Oui/Non) en 0/1.
— Découpage stratifié train/test : 70% / 30% (stabilité des proportions).
— Standardisation (z-score) pour les modèles sensibles à l'échelle (Logit, SVM).

3 Analyse descriptive : qui obtient un prêt ?

Cette section répond à la question métier : quel profil est associé à l'octroi ? . Le Tableau 1 compare les moyennes (ou proportions) entre les clients Prêt=0 et Prêt=1.

Table 1 – *Profil moyen par classe (Prêt=1 vs Prêt=0).*

Variable	Prêt=0	Prêt=1	Diff. (1-0)
Age	45.303	49.633	4.330
Experience	20.286	24.919	4.633
Revenu (milliers)	100.988	157.487	56.499
Dépense moyenne (milliers)	6151.401	10051.142	3899.741
Famille	2.496	2.481	-0.015
Education	2.011	1.991	-0.020
CEL (1=Oui)	0.252	0.642	0.390

Le tableau met en évidence des différences marquées entre les deux groupes : les clients ayant obtenu un prêt sont en moyenne plus âgés, plus expérimentés, disposent de revenus et de dépenses plus élevés, et détiennent plus fréquemment un CEL. À l'inverse, les variables Famille et Éducation présentent des écarts faibles, ce qui suggère un pouvoir explicatif limité dans l'analyse descriptive.

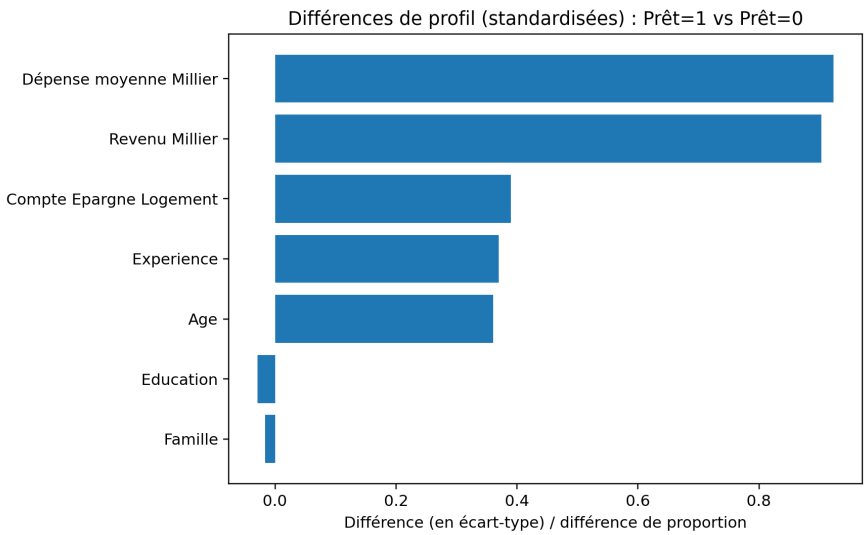

Figure 2 – *Différences standardisées entre Prêt=1 et Prêt=0. Plus la barre est éloignée de 0, plus la variable discrimine les classes.*

L'analyse des différences standardisées (Figure 2) montre que la variable présentant l'écart le plus élevé est la dépense moyenne, suivie du revenu puis du compte épargne logement (CEL).

La dépense moyenne apparaît ainsi comme la variable la plus discriminante en analyse descriptive. Elle est fortement associée à l'octroi, ce qui traduit le rôle central de la capacité financière dans la décision de crédit.

Le CEL reste toutefois une variable très différenciante : la détention d'un CEL semble refléter une relation bancaire plus établie et une meilleure connaissance du client par la banque.

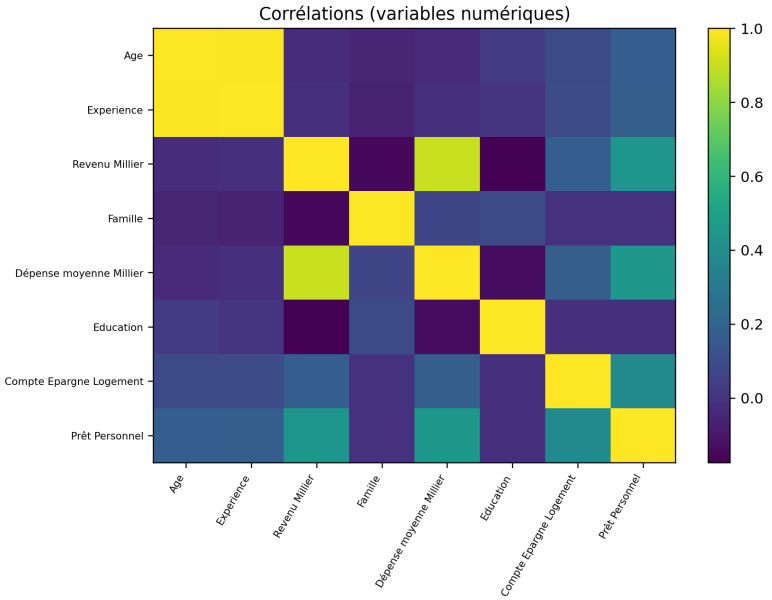

Figure 3 – *Matrice de corrélation des variables numériques.*

La matrice de corrélation révèle des corrélations positives notables entre l'âge et l'expérience, ainsi qu'entre le revenu et la dépense moyenne. Bien que cohérentes économiquement, ces relations peuvent introduire une colinéarité partielle dans le modèle, ce qui justifie l'utilisation d'une régularisation pour stabiliser les coefficients estimés.

4 Méthodes et modèles

4.1 Modèles comparés

- Régression logistique : sans régularisation, Ridge (L2), Lasso (L1).
- Arbres : Gini, Entropie.
- Ensembles : Bagging, Random Forest,AdaBoost, Gradient Boosting.
- Balanced Random Forest (librairie imbalanced-learn).
- SVM (sélection par validation croisée).

4.1 Évaluation

- AUC ROC : performance globale indépendante du seuil (discrimination).
- Accuracy, Precision, Recall au seuil 0.5 : utiles pour discuter des erreurs de décision.

4.2 Optimisation des hyperparamètres

Les hyperparamètres ont été sélectionnés par validation croisée (GridSearchCV) en maximisant l'AUC.

Table 2 – *Hyperparamètres retenus (validation croisée).*

Modèle	Paramètres principaux	Valeurs retenues
Logit Ridge (L2)	C	0.01
Random Forest	max_depth, n_estimators	8 ; 100
Bagging Logit	C, n_estimators	0.01 ; 300
AdaBoost	max_depth, learning_rate, n_estimators	5 ; 0.1 ; 100
Gradient Boosting	max_depth, learning_rate, n_estimators	5 ; 0.01 ; 400
SVM	noyau, C, gamma	rbf ; 0.1 ; 0.01

Les valeurs retenues correspondent aux paramètres maximisant l'AUC moyenne en validation croisée.

5 Résultats de performance

Les performances sur l'échantillon test sont synthétisées dans le Tableau 3.

Table 3 – *Performances sur l'échantillon test (seuil 0.5).*

Modèle	Accuracy	Precision	Recall	AUC
Logit Ridge (L2)	0.763	0.718	0.672	0.826
Bagging Logit	0.745	0.662	0.742	0.826
SVM (best)	0.762	0.723	0.657	0.826
Logit (sans régularisation)	0.764	0.719	0.673	0.825
Logit Lasso (L1)	0.765	0.720	0.673	0.825
Random Forest	0.749	0.668	0.740	0.817
GradientBoosting	0.737	0.651	0.735	0.815
AdaBoost	0.737	0.651	0.735	0.814
Bagging Arbres	0.741	0.663	0.718	0.813
Arbre (Entropie)	0.749	0.690	0.677	0.809
Arbre (Gini)	0.746	0.695	0.650	0.803
Balanced RF	0.731	0.653	0.697	0.805

Les meilleurs modèles (Logit Ridge, Bagging Logit, SVM) affichent une AUC très proche (0.826). À ce niveau, les écarts d'AUC sont faibles : le choix final doit donc intégrer des critères de gouvernance (interprétabilité, stabilité) et des critères économiques (coût des erreurs).

Les erreurs (Precision vs Recall) : Le Bagging Logit présente un Recall plus élevé : il identifie davantage de clients éligibles, au prix d'une Precision plus faible (potentiellement plus de faux positifs). Le Logit Ridge offre un compromis plus

équilibré et reste plus simple à justifier.

Les modèles d'arbres et boosting présentent des performances légèrement inférieures, ces données ne contiennent pas de relations complexes ou fortement non linéaires. Un modèle linéaire régularisé capture déjà l'essentiel de l'information utile.

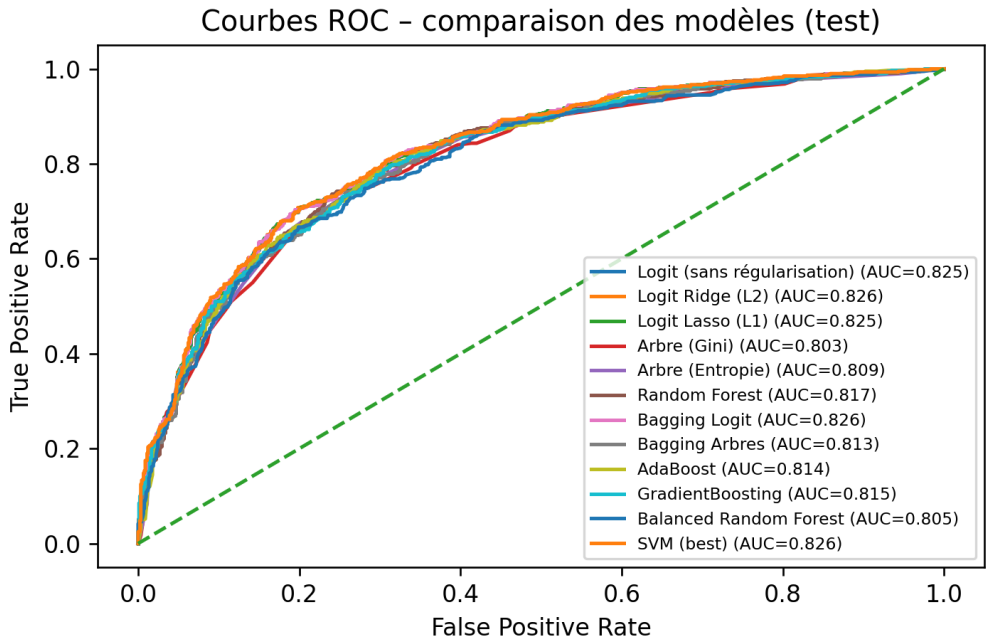

Figure 4 – *Courbes ROC sur l'échantillon test (modèles superposés).*

Les courbes ROC sont nettement situées au-dessus de la diagonale aléatoire, ce qui confirme un pouvoir discriminant réel pour l'ensemble des modèles. Toutefois, les courbes sont très proches les unes des autres, traduisant des performances similaires en termes de discrimination. Le choix final ne peut donc pas reposer uniquement sur l'AUC, mais doit intégrer des considérations d'interprétabilité et de décision économique.

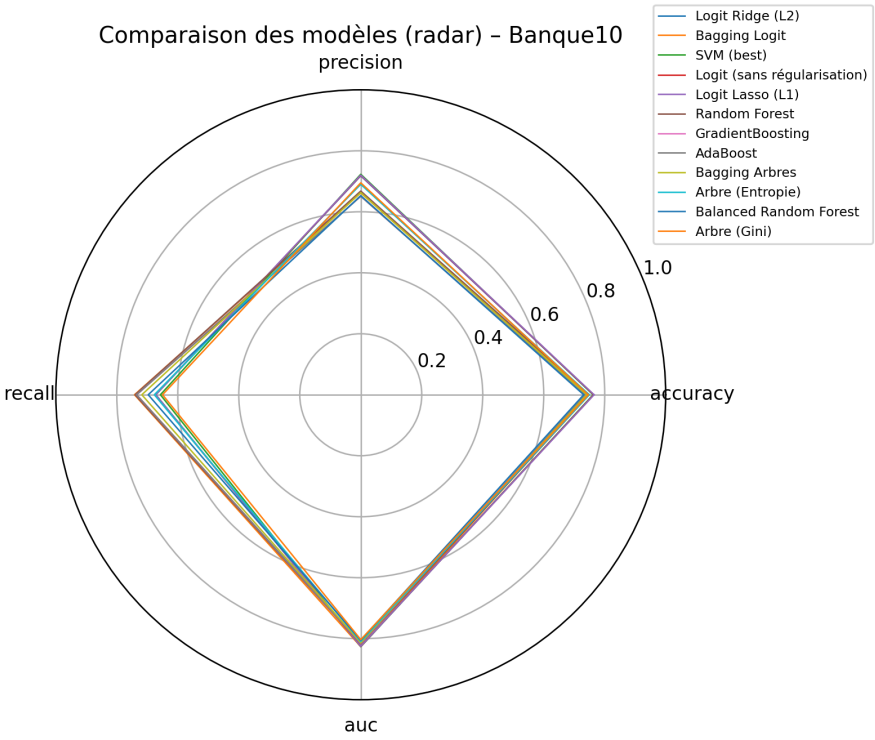

Figure 5 – *Comparaison des modèles (Accuracy, Precision, Recall, AUC).*

5.1 Les écarts sont-ils significatifs ?

Les différences d'AUC observées (au millième) sont en pratique faibles. Une partie de l'écart peut venir de la variabilité d'échantillonnage et du split train/test. Il est donc raisonnable de privilégier un modèle explicable et stable lorsque les performances sont quasi équivalentes.

5.2 Balanced Random Forest

Le Balanced Random Forest est conçu pour les jeux de données fortement déséquilibrés.

> Imbalanced-learn : La librairie imbalanced-learn propose des méthodes dédiées aux jeux de données déséquilibrés (sur-échantillonnage, sous-échantillonnage, ensembles équilibrés). Le Balanced Random Forest effectue un rééquilibrage interne à chaque bootstrap.

Dans notre cas (40/60), le déséquilibre est modéré : l'AUC du Balanced RF (0.805) reste inférieure à celle du Random Forest classique (0.817) et aux meilleurs

modèles (0.826). Le traitement spécifique du déséquilibre n'apporte donc pas de gain ici.

6 Interprétation : hiérarchie des variables (Logit Ridge)

La régression logistique Ridge combine performance et explicabilité. Sur variables standardisées, un coefficient β représente l'effet d'une variation d'un écart-type sur les log-odds. On rapporte aussi l'odds ratio $\exp\beta$.

Table 4 – *Logit Ridge : principaux coefficients (variables standardisées).*

Variable	Coef. std.	OR (par +1 σ)
CEL (1=Oui)	0.847	2.333
Dépense moyenne (milliers)	0.646	1.908
Revenu (milliers)	0.629	1.876
Experience	0.485	1.625
Education	0.189	1.208
Famille	0.062	1.064
Age	0.045	1.046

Le CEL apparaît comme la variable la plus influente du modèle, suivi de la dépense moyenne et du revenu. Les autres variables ont un impact plus modéré, ce qui suggère que la décision d'octroi repose principalement sur la relation bancaire et la capacité financière.

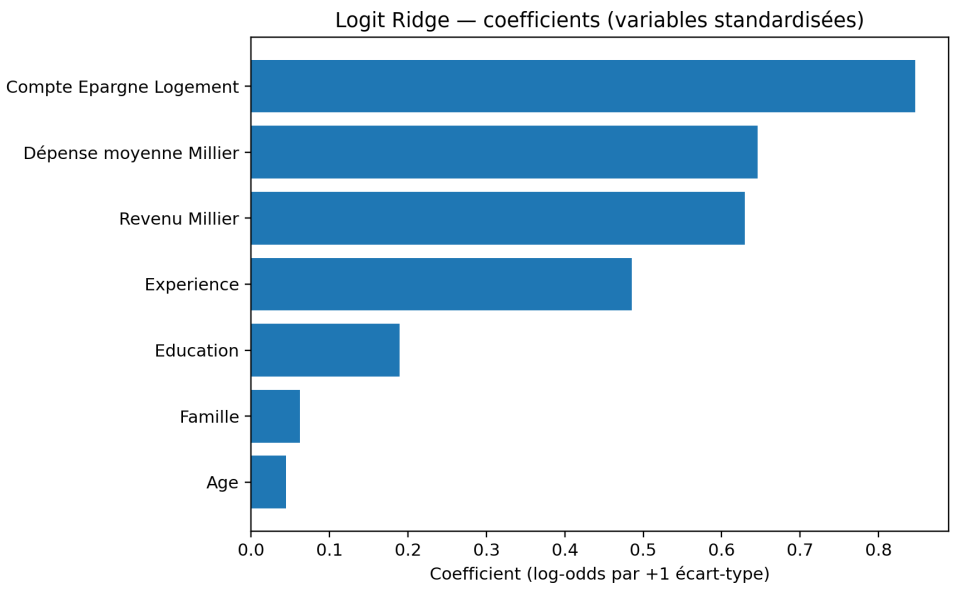

Figure 6 – *Coefficients du Logit Ridge (variables standardisées).*

Le graphique confirme visuellement que l'information prédictive est principalement portée par un nombre restreint de variables.

7 Risque bancaire : faux positifs vs faux négatifs

Une métrique peut être trompeuse si l'on ignore le coût des erreurs :

— Faux positif (FP) : accord prédit alors qu'un refus serait préférable (exposition au risque).
— Faux négatif (FN) : refus prédit alors qu'un accord était possible (coût d'opportunité).

Point clé : en pratique, le coût d'un faux positif est souvent supérieur à celui d'un faux négatif en crédit. Cela justifie de surveiller la Precision et d'ajuster le seuil pour contrôler le risque.

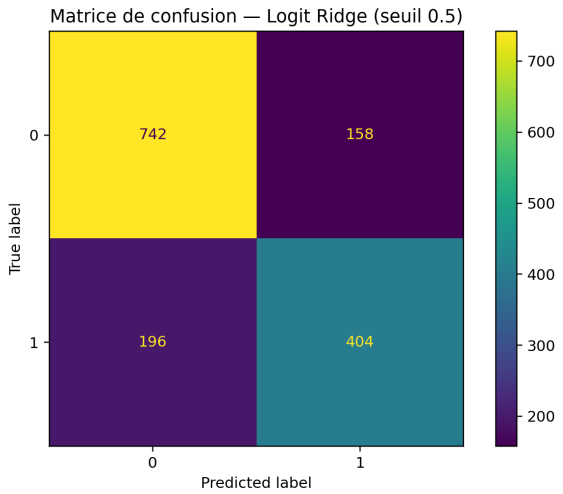

Figure 7 – *Matrice de confusion — Logit Ridge (seuil 0.5).*

La matrice de confusion montre que le modèle identifie correctement une majorité de dossiers (742 vrais négatifs et 404 vrais positifs). Les faux positifs (158) et faux négatifs (196) restent toutefois significatifs, ce qui souligne l'importance du choix du seuil dans une logique de gestion du risque. Un ajustement du seuil permettrait d'arbitrer entre exposition au risque et opportunité commerciale.

8 Choix final du modèle et recommandation

8.1 Argumentation

— Performance : les meilleurs modèles (Logit Ridge, Bagging Logit, SVM) présentent des AUC très proches (0.826). Les écarts observés restent faibles et ne permettent pas de distinguer clairement un modèle sur le seul critère de discrimination.
— Structure des résultats : les modèles linéaires pénalisés obtiennent des performances équivalentes, voire supérieures, aux méthodes plus complexes (arbres,

boosting). Cela suggère que la relation entre les variables et l'octroi est relativement bien captée par une structure linéaire régularisée.

— Gouvernance et décision économique : le Logit Ridge offre une interprétation directe des coefficients et des odds ratios, ce qui facilite la justification des décisions. Le Bagging Logit améliore légèrement le Recall mais peut accroître le nombre de faux positifs selon le seuil retenu.

8.2 Recommandation

Modèle recommandé : Logit Ridge (L2). Il combine un bon niveau de performance (AUC 0.826), une stabilité satisfaisante et une interprétabilité adaptée à un contexte bancaire où la justification des décisions est essentielle.

Alternative : Bagging Logit. Ce modèle peut être envisagé si l'objectif prioritaire est de réduire les faux négatifs dans une logique plus commerciale, en acceptant un risque légèrement plus élevé de faux positifs.

9 Conclusion

L'analyse met en évidence que l'octroi de prêt est principalement associé à la capacité financière du client (revenu, dépense moyenne) ainsi qu'à la relation bancaire mesurée par la détention d'un CEL.

La comparaison des modèles montre des performances très proches en termes de discrimination. Dans ce contexte, le Logit Ridge constitue le meilleur compromis entre performance prédictive, stabilité et interprétabilité.

Le modèle retenu permet ainsi de fournir une probabilité d'octroi exploitable dans un cadre décisionnel cohérent avec les exigences d'un établissement bancaire.

Code

```python
# -*- coding: utf-8 -*-
"""
Created on Fri Feb  1 11:39:13 2026

@author: Laury
"""

import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

from sklearn.metrics import roc_curve, roc_auc_score


# ========================
# 0) Imports
# ========================

# Fonctions d'évaluation et graphiques fournies
# centralisent le calcul des métriques et les affichages standardisés
from fonction1 import (
    evaluation_modele_logit,
    evaluation_modele,
    evaluation_modele_bagging,
    graph_imp_var,
    graph_variable_bagging,
    graph_variable_boosting,
    metriques_evaluation
)

# Fonction de comparaison globale des modèles d'ensemble
```

```python
from fonction_comparer import comparer_modeles

# Modèle forêt équilibrée (utile si classes déséquilibrées)
from imblearn.ensemble import BalancedRandomForestClassifier

# Indicateurs logiques pour activer certaines parties du script
HAS_COMPARER = True
HAS_IMBLEARN = True


# ========================
# 1) Paramètres & chemins
# ========================

# Fichiers sources (données d'entraînement et nouveaux clients)
TRAIN_PATH = "Banque10.csv"
NEW_PATH   = "Banque_Classification.csv"

# Dossier principal de sortie pour tous les résultats
OUT_DIR = "rapport_banque10_outputs_aligne"

# Sous-dossier pour les figures (créé automatiquement s'il n'existe pas)
FIG_DIR = os.path.join(OUT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# Graine aléatoire pour garantir la reproductibilité
RANDOM_STATE = 42


# ========================
# 2) Chargement + nettoyage
# ========================
train = pd.read_csv(TRAIN_PATH, sep=";", encoding="cp1252")  # lecture base d'entranement
new   = pd.read_csv(NEW_PATH,   sep=";", encoding="cp1252")  # lecture base prdire

train.columns = [c.strip() for c in train.columns]  # suppression espaces noms colonnes
new.columns   = [c.strip().replace("", "").strip() for c in new.columns]  # nettoyage symbole

# Harmonisation de certains noms
rename = {}
for c in new.columns:
    if "Revenu Millier" in c:
        rename[c] = "Revenu Millier"  # uniformisation nom variable revenu
    if "Dépense moyenne Millier" in c or "Depense moyenne Millier" in c:
```

```
        rename[c] = "Dpense moyenne Millier"  # uniformisation nom variable dpense
new = new.rename(columns=rename)

# Recodage Oui/Non -> 1/0
bin_map = {"Oui": 1, "Non": 0, "OUI": 1, "NON": 0, "oui": 1, "non": 0}
train["Compte Epargne Logement"] = train["Compte Epargne Logement"].map(bin_map).astype(int)  # variable explicative binaire
train["Prt Personnel"]           = train["Prt Personnel"].map(bin_map).astype(int)  # variable cible
new["Compte Epargne Logement"]   = new["Compte Epargne Logement"].map(bin_map).astype(int)

# Séparation X / y
X = train.drop(columns=["Prêt Personnel", "ID"])  # matrice explicative
y = train["Prêt Personnel"]  # cible
newX = new[X.columns]  # alignement strict colonnes

# Split stratifié
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y  # conservation proportion classes
)

# Standardisation (Logit & SVM)
features = list(X.columns)
prep = ColumnTransformer([("num", StandardScaler(), features)], remainder="drop")  # scaling numrique

# CV
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)  # validation croise stratifie


# ========================
# 3) Outils internes (ROC + stockage métriques)
# ========================
rows = []          # stockage métriques finales
roc_store = {}     # stockage courbes ROC
models = {}        # stockage objets modèles entraînés

def get_proba(model, X_):  # récupère score utilisable pour AUC
    "Retourne une probabilité (ou proxy) pour ROC/AUC."
    if hasattr(model, "predict_proba"):  # cas standard
        return model.predict_proba(X_)[:, 1]
    if hasattr(model, "decision_function"):  # cas SVM sans proba
        scores = model.decision_function(X_)
```

```
        return (scores - scores.min()) / (scores.max() - scores.min() + 1e-9)  # normalisation
    pred = model.predict(X_)
    return np.asarray(pred, dtype=float)

def store_roc(name, model, X_test_, y_test_):  # calcule et enregistre ROC
    proba = get_proba(model, X_test_)
    fpr, tpr, _ = roc_curve(y_test_, proba)
    auc_ = roc_auc_score(y_test_, proba)
    roc_store[name] = (fpr, tpr, auc_)

def add_metrics(name, model):  # ajoute ligne dans tableau final
    "Ajoute une ligne de mtriques via la fonction du prof (accuracy/precision/recall/auc)."
    acc, pre, rec, auc_ = metriques_evaluation(model, X_test, y_test)
    rows.append([name, acc, pre, rec, auc_])

# =========================
# 4) Estimation des modèles
# =========================

# --- (A) LOGIT
print("
=== LOGIT ===")  # section régression logistique
logit = Pipeline([("prep", prep),
                  ("model", LogisticRegression(penalty=None, solver="lbfgs", max_iter=3000))])  # logit sans rgularisation
logit.fit(X_train, y_train)  # entraînement
models["Logit (sans régularisation)"] = logit  # stockage modèle

evaluation_modele_logit(logit, X_test, y_test, "Logit (sans rgularisation)")  # valuation test
store_roc("Logit (sans régularisation)", logit, X_test, y_test)  # stockage ROC
add_metrics("Logit (sans régularisation)", logit)  # ajout métriques tableau

# Ridge
ridge_pipe = Pipeline([("prep", prep),
                       ("model", LogisticRegression(penalty="l2", solver="lbfgs", max_iter=3000))])  # logit L2
ridge_grid = {"model__C": [0.01, 0.1, 1, 10, 100]}  # grille hyperparamètre
ridge_gs = GridSearchCV(ridge_pipe, ridge_grid, scoring="roc_auc", cv=cv, n_jobs=-1, verbose=0)  # validation croise
ridge_gs.fit(X_train, y_train)
ridge = ridge_gs.best_estimator_  # meilleur modèle
models["Logit Ridge (L2)"] = ridge

evaluation_modele_logit(ridge, X_test, y_test, f"Logit Ridge (L2) best={ridge_gs.best_params_}")
```

```python
store_roc("Logit Ridge (L2)", ridge, X_test, y_test)
add_metrics("Logit Ridge (L2)", ridge)

# Lasso
lasso_pipe = Pipeline([("prep", prep),
                       ("model", LogisticRegression(penalty="l1", solver="liblinear", max_iter=3000))])  # logit L1
lasso_grid = {"model__C": [0.01, 0.1, 1, 10, 100]}
lasso_gs = GridSearchCV(lasso_pipe, lasso_grid, scoring="roc_auc", cv=cv, n_jobs=-1, verbose=0)
lasso_gs.fit(X_train, y_train)
lasso = lasso_gs.best_estimator_
models["Logit Lasso (L1)"] = lasso

evaluation_modele_logit(lasso, X_test, y_test, f"Logit Lasso (L1) best={lasso_gs.best_params_}")
store_roc("Logit Lasso (L1)", lasso, X_test, y_test)
add_metrics("Logit Lasso (L1)", lasso)


# --- (B) ARBRES
print("
=== ARBRES ===")  # section arbres de décision
tree_grid = {"max_depth": [3, 5, 8, None], "min_samples_leaf": [1, 2, 5, 10]}  # grille hyperparamtres

for crit, label in [("gini", "Arbre (Gini)"), ("entropy", "Arbre (Entropie)")]:  # deux critres de division
    tree = DecisionTreeClassifier(criterion=crit, random_state=RANDOM_STATE)  # initialisation arbre
    gs = GridSearchCV(tree, tree_grid, scoring="roc_auc", cv=cv, n_jobs=-1, verbose=0)  # recherche optimale
    gs.fit(X_train, y_train)
    best_tree = gs.best_estimator_  # meilleur arbre
    models[label] = best_tree  # stockage modèle

    evaluation_modele(best_tree, X_test, y_test, f"{label} best={gs.best_params_}")  # valuation test
    store_roc(label, best_tree, X_test, y_test)  # stockage ROC
    add_metrics(label, best_tree)  # ajout métriques

# Importance variables arbres

plt.figure()  # nouvelle figure
graph_imp_var(models["Arbre (Gini)"], features, print_values=False)  # importance variables
plt.tight_layout()
```

```python
plt.savefig(os.path.join(FIG_DIR, "importance_arbre_gini.png"), dpi=220)  # sauvegarde image
plt.close()

# --- (C) ENSEMBLES
print("=== ENSEMBLES ===")  # section méthodes d'ensemble
if HAS_COMPARER:
    print("-> Utilisation de comparer_modeles()")
    try:
        out = comparer_modeles(X_train, X_test, y_train, y_test, a=1, b=1, c=1)  # lance comparaison

        if isinstance(out, tuple) and len(out) >= 3 and isinstance(out[2], dict):  # cas (.., df, dict)
            df_ens, dict_ens = out[1], out[2]
        elif isinstance(out, tuple) and len(out) == 2 and isinstance(out[1], dict):  # cas (df, dict)
            df_ens, dict_ens = out[0], out[1]
        elif isinstance(out, dict):  # cas dict seul
            df_ens, dict_ens = None, out
        else:
            raise ValueError("Signature inattendue de comparer_modeles().")  # protection

        if df_ens is not None:
            df_ens.to_csv(os.path.join(OUT_DIR, "comparaison_ensembles_prof.csv"), index=False)  # sauvegarde rsultats

        for name, est in dict_ens.items():  # boucle sur modèles renvoyés
            models[name] = est  # stockage estimateur
            evaluation_modele(est, X_test, y_test, name)  # évaluation test
            store_roc(name, est, X_test, y_test)  # stockage ROC
            add_metrics(name, est)  # ajout métriques

    except Exception as e:
        print("!! comparer_modeles a chou -> fallback rapide. Erreur :", repr(e))  # diagnostic
        HAS_COMPARER = False  # bascule fallback

if not HAS_COMPARER:
    print("-> Fallback : entranement simple des ensembles.")  # version autonome

    bag = BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=RANDOM_STATE),
        n_estimators=200, max_samples=0.9, max_features=1.0, random_state=RANDOM_STATE
```

```
    )
    bag.fit(X_train, y_train)
    models["Bagging (arbres)"] = bag
    evaluation_modele_bagging(bag, X_test, y_test, "Bagging (arbres)")
    store_roc("Bagging (arbres)", bag, X_test, y_test)
    add_metrics("Bagging (arbres)", bag)

    rf = RandomForestClassifier(n_estimators=600, max_features="sqrt", random_state=RANDOM_STATE, n_jobs=-1)  # fort alatoire
    rf.fit(X_train, y_train)
    models["Random Forest"] = rf
    evaluation_modele(rf, X_test, y_test, "Random Forest")
    store_roc("Random Forest", rf, X_test, y_test)
    add_metrics("Random Forest", rf)

    ada = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
        n_estimators=400, learning_rate=0.1, random_state=RANDOM_STATE, algorithm="SAMME"
    )
    ada.fit(X_train, y_train)
    models["AdaBoost"] = ada
    evaluation_modele(ada, X_test, y_test, "AdaBoost")
    store_roc("AdaBoost", ada, X_test, y_test)
    add_metrics("AdaBoost", ada)

    gb = GradientBoostingClassifier(
        n_estimators=600, learning_rate=0.05, max_depth=3, subsample=0.9, random_state=RANDOM_STATE
    )
    gb.fit(X_train, y_train)
    models["GradientBoosting"] = gb
    evaluation_modele(gb, X_test, y_test, "GradientBoosting")
    store_roc("GradientBoosting", gb, X_test, y_test)
    add_metrics("GradientBoosting", gb)

    plt.figure()
    graph_variable_boosting(gb, features, top_n=10)  # importance variables boosting
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "importance_boosting.png"), dpi=220)  # sauvegarde figure
    plt.close()

# --- (D) BalancedRandomForestClassifier (imblearn)
print("
=== BALANCED RANDOM FOREST ===")  # fort alatoire avec rquilibrage automatique
```

```python
if HAS_IMBLEARN:
    brf = BalancedRandomForestClassifier(
        n_estimators=600, max_features="sqrt", random_state=RANDOM_STATE, n_jobs=-1  # hyperparamtres principaux
    )
    brf.fit(X_train, y_train)  # entraînement
    models["Balanced Random Forest"] = brf  # stockage modèle
    evaluation_modele(brf, X_test, y_test, "Balanced Random Forest")  # valuation test
    store_roc("Balanced Random Forest", brf, X_test, y_test)  # stockage ROC
    add_metrics("Balanced Random Forest", brf)  # ajout métriques
else:
    print("!! imblearn non install -> BalancedRandomForest ignor. Pour l'activer : pip install imbalanced-learn")  # message si package absent

# --- (E) SVM
print("
=== SVM ===")  # section Support Vector Machine
svm_pipe = Pipeline([("prep", prep), ("model", SVC(probability=True, random_state=RANDOM_STATE))])  # pipeline SVM + scaling
svm_grid = [
    {"model__kernel": ["linear"], "model__C": [0.1, 1, 10, 100]},  # noyau linaire
    {"model__kernel": ["rbf"], "model__C": [0.1, 1, 10, 100], "model__gamma": ["scale", 0.1, 0.01]},  # noyau gaussien
    {"model__kernel": ["poly"], "model__C": [0.1, 1, 10], "model__degree": [2, 3], "model__gamma": ["scale"]}  # noyau polynomial
]
svm_gs = GridSearchCV(svm_pipe, svm_grid, scoring="roc_auc", cv=cv, n_jobs=-1, verbose=0)  # recherche hyperparamtres
svm_gs.fit(X_train, y_train)
svm_best = svm_gs.best_estimator_  # meilleur SVM
models["SVM (best)"] = svm_best  # stockage modèle

evaluation_modele(svm_best, X_test, y_test, f"SVM best={svm_gs.best_params_}")  # valuation test
store_roc("SVM (best)", svm_best, X_test, y_test)  # stockage ROC
add_metrics("SVM (best)", svm_best)  # ajout métriques

# ========================
# 5) Tableau + radar + ROC combinées
# ========================
comparaison = pd.DataFrame(rows, columns=["model", "accuracy", "precision", "recall", "auc"]).drop_duplicates("model")  # cration tableau mtriques
comparaison = comparaison.sort_values("auc", ascending=False).reset_index(drop=True)  # tri par performance AUC
comparaison.to_csv(os.path.join(OUT_DIR, "metrics_all_models.csv"), index=False)  # sauvegarde CSV
```

```python

# Radar (style cours)
metrics_cols = ["accuracy", "precision", "recall", "auc"]  # mtriques utilises
N = len(metrics_cols)
angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist()  # angles radar
angles += angles[:1]  # fermeture du polygone

fig = plt.figure(figsize=(7, 7))
ax = plt.subplot(111, polar=True)  # graphique polaire

for i in range(len(comparaison)):
    values = comparaison.loc[i, metrics_cols].tolist()  # valeurs modèle i
    values += values[:1]  # fermeture
    ax.plot(angles, values, linewidth=0.8, label=comparaison.loc[i, "model"])  # trac ligne
    ax.fill(angles, values, color="white", alpha=0.08)  # remplissage léger

ax.set_xticks(angles[:-1])
ax.set_xticklabels(metrics_cols)
ax.set_ylim(0, 1)  # échelle métriques
plt.legend(loc="upper right", bbox_to_anchor=(1.35, 1.15), fontsize=7)
plt.title("Comparaison des modèles (radar) - Banque10")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "radar_metrics.png"), dpi=240)  # sauvegarde radar
plt.close()

# ROC combinées
plt.figure()
for name, (fpr, tpr, auc_) in roc_store.items():  # boucle sur modèles
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc_:.3f})")  # tracé courbe ROC
plt.plot([0, 1], [0, 1], linestyle="--")  # diagonale aléatoire
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Courbes ROC - comparaison des modèles (test)")
plt.legend(fontsize=7)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "roc_compare.png"), dpi=240)  # sauvegarde ROC
plt.close()

# Sauvegarde ROC brut
roc_json = {k: {"fpr": list(map(float, v[0])), "tpr": list(map(float, v[1])), "auc": float(v[2])} for k, v in roc_store.items()}  # export brut ROC
with open(os.path.join(OUT_DIR, "roc_all_models.json"), "w", encoding="utf-8") as f:
    json.dump(roc_json, f, ensure_ascii=False)  # écriture fichier JSON

```

```
# ========================
# 6) Meilleur modèle + prédictions nouveaux clients
# ========================
best_name = comparaison.loc[0, "model"]  # meilleur modèle selon tri AUC
best_model = models[best_name]  # récupération estimateur
print("
>>> Meilleur modèle (trié par AUC) :", best_name)  # affichage

proba_new = get_proba(best_model, newX)  # probabilit prt=1 sur nouveaux clients
pred_new = (proba_new > 0.5).astype(int)  # règle de décision (seuil 0.5)

pred_df = new.copy()  # base originale nouveaux clients
pred_df["Proba_Pret_Oui"] = proba_new  # ajout proba
pred_df["Prediction_Pret_Oui"] = pred_new  # ajout classe prédite
pred_df.to_csv(os.path.join(OUT_DIR, "predictions_Banque_Classification.csv"), index=False)  # export rsultats

print("
Terminé. Fichiers produits dans :", OUT_DIR)  # récap dossier
print(comparaison.head(10))  # top 10 modèles

```